AWS MLA-C02 • BLA 04 • PART 2

Provisioning and Configuring ML and AI Resources

Deployment and Orchestration of ML and AI Workflows

02

THEORY-ONLY
LEARNING SERIES

Deploy • Provision • Automate



Provisioning balances performance and cost

Capacity choices should follow how predictable the workload is.

On-demand

Pay for what you use, with no long-term commitment.

- Unpredictable load
- Bedrock per-token pricing
- Simple to start

Provisioned

Reserve capacity for steady, high-volume traffic.

- Bedrock Provisioned Throughput
- SageMaker Savings Plans
- Consistent performance

Spot capacity

Use spare EC2 capacity for interruptible training jobs.

- Managed Spot Training
- Large savings vs on-demand
- Needs checkpointing



Provision resources with code, not clicks

Repeatable automation keeps environments consistent.

Infrastructure as code

- AWS CloudFormation templates define stacks
- AWS CDK builds stacks in Python or TypeScript
- Stack outputs and exports share values
- Parameter Store passes settings between stacks

Programmatic deployment

- SageMaker Python SDK: model.deploy()
- Boto3: model, endpoint config, endpoint
- AWS CLI for scripts and pipelines
- Step Functions and EventBridge orchestrate



Containers package ML and AI workloads

Choose the least custom option that supports your code.

Prebuilt containers

AWS Deep Learning Containers for common frameworks and LLM serving.

- Maintained by AWS
- Fastest path to deploy
- Add your own script

Extended containers

Start from a prebuilt image and add dependencies.

- Add libraries in a Dockerfile
- Keep AWS base updates
- Push to Amazon ECR

Bring your own

Build a fully custom image for special runtimes.

- Serve /invocations and /ping
- Listen on port 8080
- Scan and tag versions



Endpoints can run inside a private network

VPC settings control how traffic reaches the model and leaves it.

VPC configuration

Attach subnets and security groups to the model.

- Private subnets, multiple AZs
- Security groups limit ports
- Reach private data stores

Interface endpoints

AWS PrivateLink keeps API calls off the public internet.

- Endpoint for SageMaker Runtime
- S3 gateway endpoint
- Endpoint policies

Network isolation

Blocks all outbound network calls from the container.

- Protects sensitive data
- No internet dependencies
- Regulated workloads



Auto scaling depends on the right metric

The metric should track what actually limits the endpoint.

Invocation metrics

Target tracking on invocations per instance.

- Default for real-time
- Load test to set target
- Set min and max counts

Resource metrics

CPU, memory, and GPU utilization show saturation.

- GPUUtilization
- GPUMemoryUtilization
- Concurrent requests

Queue metrics

Asynchronous endpoints scale on queue backlog.

- Backlog size per instance
- Scale to zero when empty
- Step scaling to wake up



Amazon Bedrock Knowledge Bases power RAG

Indexing quality decides retrieval quality.

1 Ingest
Connect Amazon S3 or other data sources

›

2 Chunk
Fixed, semantic, or hierarchical splits

›

3 Embed
Create vectors with an embeddings model

›

4 Store
OpenSearch Serverless, Aurora, or S3 Vectors

›

5 Retrieve
Top-k, hybrid search, and reranking

Retrieval pipelines should match business needs: precise answers favor small chunks and reranking; broad questions favor larger context.



Agentic workloads need state and scale

Agents must remember context and run reliably under load.

Agent state

Keep conversation and task context across turns.

- Session attributes
- AgentCore Memory
- DynamoDB for custom state

GPU scaling

Size accelerators for FM and agent traffic.

- Right-size GPU instances
- Scale components to zero
- Watch GPU memory

Agent infrastructure

Deploy runtimes, tools, and identity together.

- AgentCore Runtime
- Gateway exposes MCP tools
- Least-privilege IAM roles



A provisioning checklist

Good infrastructure is repeatable, private, and right-sized.

1. Match on-demand, provisioned, or Spot capacity to the workload.
2. Define every stack and deployment in code.
3. Use the least custom container that works.
4. Place endpoints in a VPC with private access.
5. Scale on metrics that reflect the real bottleneck.
6. Index knowledge bases and manage agent state deliberately.

